GUI LIBRARY SYSTEM

HAMS ALI

CS 2132

DR. MOHAMMAD NAUMAN

PROJECT DISCERPTION

- The Simple Library Management System simplifies library operations, allowing easy book and patron management. It ensures data accuracy, offers intuitive interfaces, and uses Object-Oriented Programming principles. Future plans involve database integration for scalability and data persistence.

THIS PROGRAM WILL SOLVE :

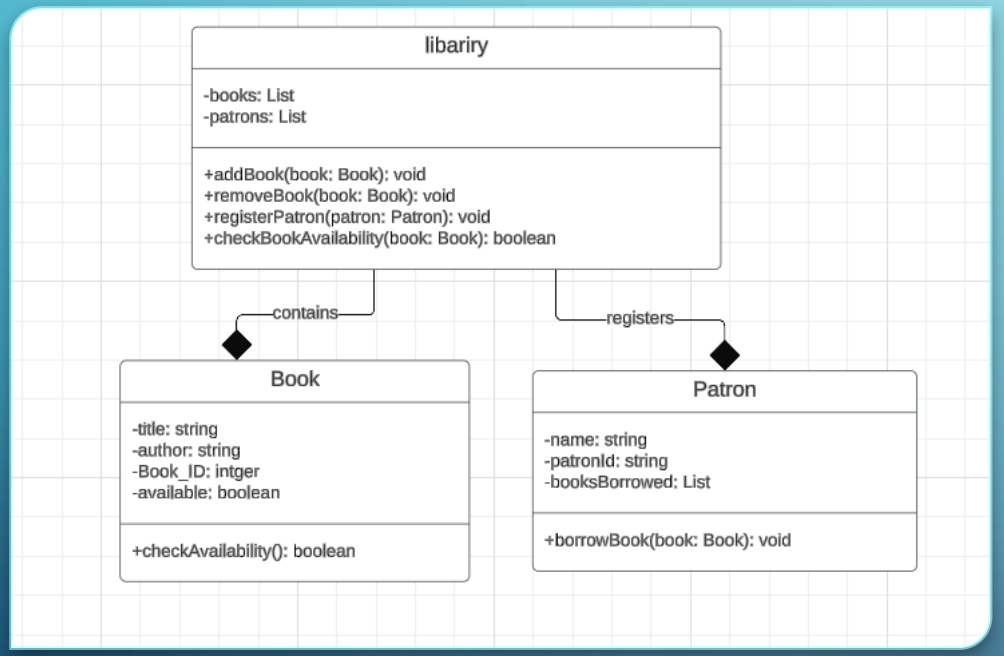
1. SORTING ALGORITHM : used to arrange books alphabetically by title or author, enabling efficient browsing for patrons.

2. BOOK RECOMMENDATION ALGORITHM : suggests books to patrons based on their interests and borrowing history, enhancing their library experience.

benefits of the library management system program

- Efficient book management: Add, remove, and check book availability.
- Patron registration: Keep track of patrons borrowing books.
- Easy-to-use interface for both librarians and patrons.
- Scalable to accommodate various library sizes.
- Error handling for smooth operation.

IMPLEMENT CLASSES FOR THE FOLLOWING ENTITIES



- Book: Represents a book with attributes such as title, author, Book ID , etc.
- Patron: Represents a library patron with attributes such as name, ID, etc.
- Library: Represents the library itself, responsible for managing books, patrons,.

Implements the class before the GUI

```java
import java.util.ArrayList;
import java.util.List;

public class Library {

    public List<Book> Books;
    public String name;

    public Library(String name) {
        this.name = name;
        this.Books = new ArrayList<>();
    }
    public void addBook(Book book) {
        this.Books.add(book);
        System.out.println("Book '" + book.getTitle() + "' added to library '" + this.name + "'.");
    }

    public void removeBook(Book book) {
        if (this.Books.remove(book)) {
            System.out.println("Book '" + book.getTitle() + "' removed from library '" + this.name +
"'.");
        } else {
            System.out.println("Book '" + book.getTitle() + "' not found in library '" + this.name +
"'.");
        }
    }

    public void add_Patron(Patron patron) {
        System.out.println("Patron '" + patron.getName() + "' registered at library '" + this.name +
"'.");
    }

    public void checkAvailability(Book book) {
        if (this.Books.contains(book)) {
            System.out.println("Book '" + book.getTitle() + "' is available in library '" + this.name +
"'.");
        } else {
            System.out.println("Book '" + book.getTitle() + "' is not available in library '" + this.name
+ "'.");
        }

}
```

```java
class Book extends Library {
    private String title;
    private String author;
    private int bookId;
    protected boolean available;

    public Book(String title, String author, int bookId)
{       this.title = title;
        this.author = author;
        this.bookId = bookId;
        this.available = true;
    }

    public boolean checkAvailability() {
        return available;
    }
}
```

```java
class Patron extends Library {
    private String name;
    private String patronId;
    private List<Book> booksBorrowed;

    public Patron(String name, String patronId) {
        this.name = name;
        this.patronId = patronId;
        this.booksBorrowed = new ArrayList<>();
    }

    public void borrowBook(Book book) {
        if (book.checkAvailability()) {
            booksBorrowed.add(book);
            book.available = false;
        } else {
            System.out.println("Sorry, the book is not
available.");
    }
}
```

MAIN CLASS

•The Main class will collect all the classes and create an object. Here we notice that we have created a new object from the library.

```java
public class main {


    public static void main(String[] args) {
         // Create a library
        Library library = new Library("Central Library");

        // Add some books to the library
        Book book1 = new Book("Harry Potter", "J.K. Rowling");
        Book book2 = new Book("To Kill a Mockingbird", "Harper Lee");
        library.addBook(book1);
        library.addBook(book2);

        // Create a patron and register at the library
        Patron patron = new Patron("John Doe", "123");
        library.registerPatron(patron);

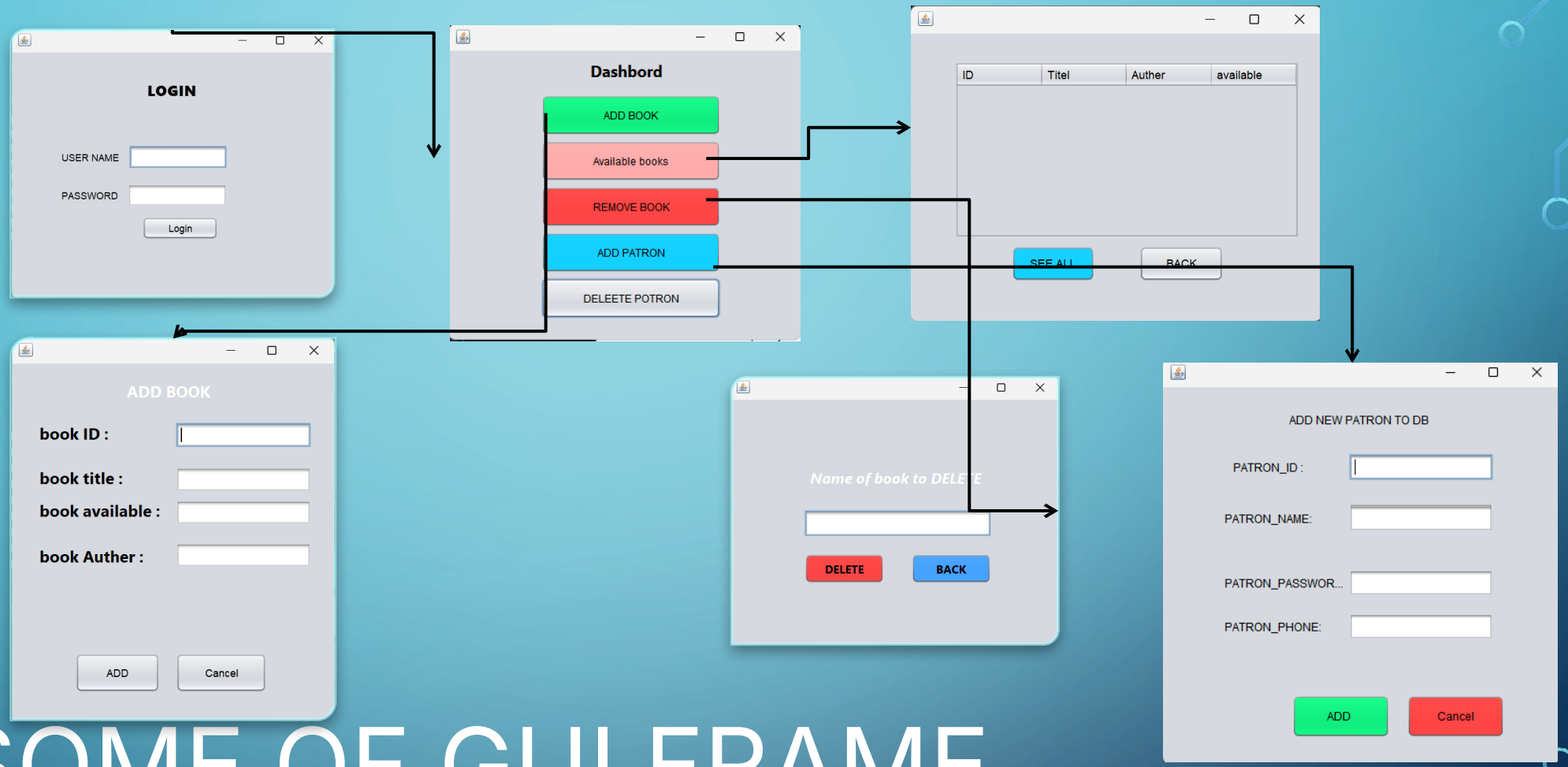
        // Check availability of a book
        library.checkAvailability(book1);

        // Remove a book from the library
        library.removeBook(book1);

        // Try to remove a book that doesn't exist
        library.removeBook(book1);

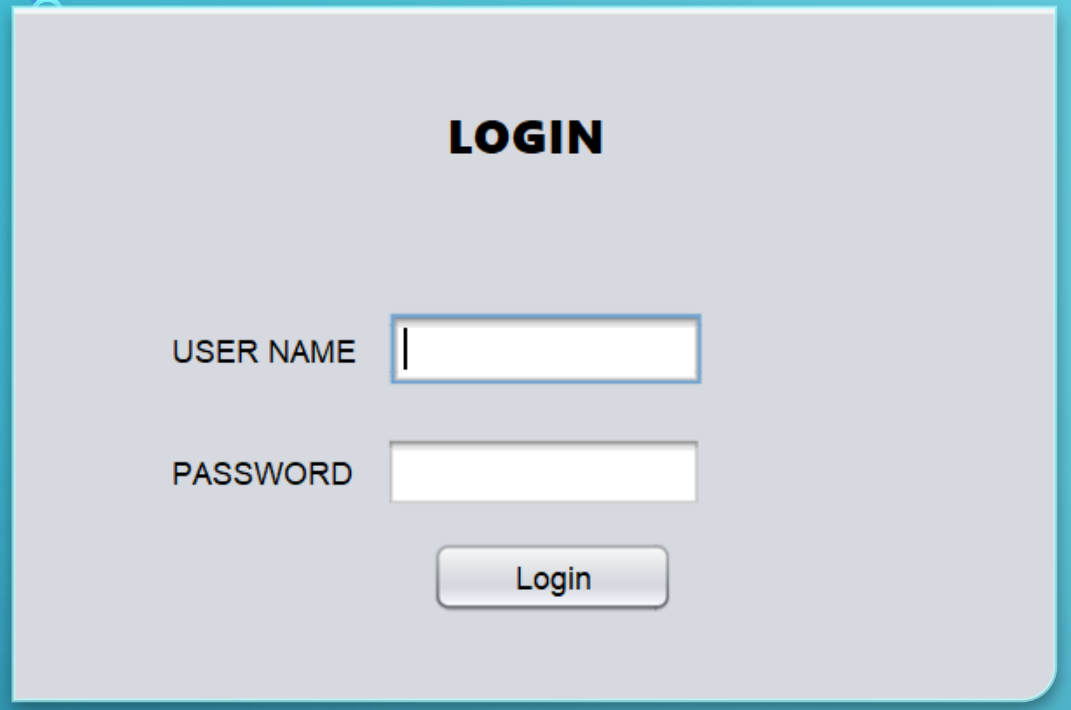
        // Patron borrows a book
        patron.borrowBook(book2);
    }
    }

}
```



SOME OF GUI FRAME DESAIN

some pictures of graphical interfaces and how they are interconnected




```
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {

        String url_db = "jdbc:mysql//localhost:3306/library?userSSL=false";
        String user_db = "root";
        String password_db = "";
        String user_faild = username1.getText();
        String pasword_faild = new String(password.getText());

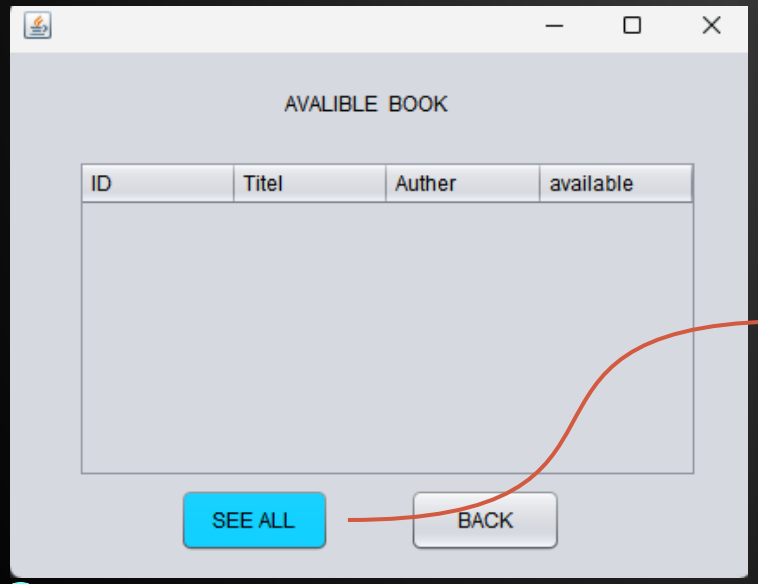

        String Sql = "SELECT admin_name , admin_password FROM admin WHERE admin_name='" + user_faild + "'
AND admin_password='" + pasword_faild + "'; ";
        try {
            Connection con_db = DriverManger.getConnection(url_db, user_db, password_db);
            Statment stm = con_db.createStatment();
            Resultest rs = stm.executeQuery(Sql) //True or false
            if (rs.next()) {
                String paswd = rs.getString("admin_password") // column name
                if (rs.equals(paswd)) {
                    dashbord ds = new dashbord();
                    ds.setVisible(true);
                    //this will close the login page
                    this.dispose();

                } else {
                    System.err.println("Eror IN  CONACTION ");
                }
            }
        } catch (Exception e) {
            System.out.println(e);
        }
```

LOGIN FUNCTIONALITY :

WHEN THE USER CLICKS ON LOGIN, IT TAKES THE TWO VALUES FROM THE USER AND EXECUTES THE QUERY THAT VERIFIES THE ADMIN NAME AND PASSWORD IN THE DATABASE.

Book Available functionality

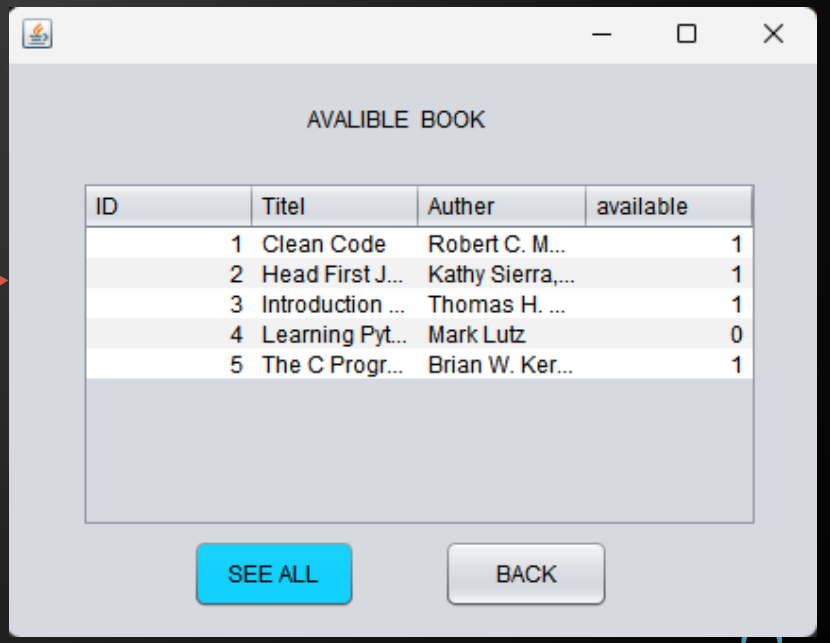



```java
private void fichActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:
    jTable1 table = jTable1.getModel();
    String url_db = "jdbc:mysql//localhost:3306/library";
    String user_db = "root";
    String password_db = "";
    String sql = "SELECT * FROM BOOK";
    try {
        Connection con_db = DriverManger.getConnection(url_db, user_db, password_db);
        Statment stm = con_db.createStatment();
        Resultest rs = stm.executeQuery(Sql); //True or false
       while(rs.next()) {
            String book_isbn = rs.getString("book_isbn"); // column name
            String book_title = rs.getString("book_title"); // column name
            String book_auther = rs.getString("book_auther"); // column name
            String book_available = rs.getString("book_available"); // column name
//              add the new row in the table
            table.addRow(new Object[] {book_isbn ,book_title,book_auther,book_available});

        }
       rs.close();
       stm.close();

    } catch (Exception e) {
        System.out.println(e);
    }
}
```

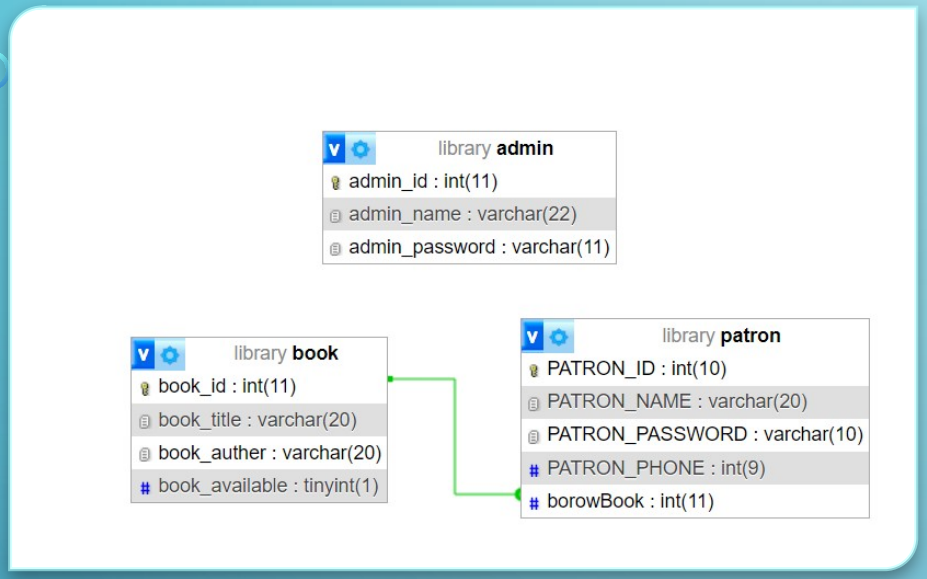


To find the available books and their details, we click on the button and then we fetch the details of the books from the database from the books table, then we store the data in the table object .



When we press the delete button, the program will see that it takes the value from the field and searches the database for the book that needs to be deleted. If it finds it, it will delete it.

BOOK DELETE FUNCTIONALITY





DATABASE TABLES


THANK YOU